

POLITECNICO DI TORINO



Lab 3

Exercise 2 - Gesture-based Visual Control

Music and multimedia streaming over the Internet

Academic year 2025/2026

Full Name : MEHRAN BAYAT

Student Number : S320636.

Cristina Rottondi, Leonardo Severi, Massimiliano Zanoni

Version: 2026

©2025-26

README

Lab 3 Exercise 2: Gesture-based Visual Control

Hand Gesture Regression Using MediaPipe, OSC, Wekinator, and Processing
RGB Visualization

Final_Report.md

Contents

1 Title	3
2 Abstract	3
3 Exercise Objective	3
4 System Overview	3
5 Software and Libraries Used	4
6 Python Input Script	4
6.1 Purpose of the Python Script	5
6.2 MediaPipe Hand Detection	5
6.3 Landmark Coordinate Extraction	5
6.4 Landmark Modes	5
6.5 OSC Output from Python to Wekinator	6
7 Wekinator Regression Project	6
7.1 Purpose of Wekinator	6
7.2 Wekinator Input Configuration	6
7.3 Wekinator Output Configuration	7
7.4 Training Target Colors	7
8 Processing Output Sketch	8
8.1 Purpose of the Output Sketch	8
8.2 OSC Input to Processing	8
8.3 Mapping Wekinator Outputs to RGB	8
9 Training Methodology	8
9.1 Training Concept	8
9.2 Training Workflow	9
9.3 Recorded Training Data Summary	9
10 Experiment 1 - 21 Landmarks and 63 Inputs	9
10.1 Configuration	9
10.2 Training Data	10
10.3 Result	10
11 Experiment 2 - 10 Landmarks and 30 Inputs	10
11.1 Configuration	10
11.2 Training Data	11

11.3 Result	11
12 Experiment 3 - 3 Landmarks and 9 Inputs	11
12.1 Configuration	11
12.2 Training Data	12
12.3 Result	12
13 Detailed Recorded Data Comparison	12
13.1 Input and Output Comparison	12
13.2 Training Example Distribution	13
13.3 Training Duration and Recording Rate	13
13.4 Wekinator Data and Model File Sizes	13
14 Performance Analysis and Discussion	13
14.1 Regression Behavior	13
14.2 Experiment 1: 21 Landmarks	14
14.3 Experiment 2: 10 Landmarks	14
14.4 Experiment 3: 3 Landmarks	14
14.5 Overall Comparison	14
15 Testing and Verification	14
15.1 Test 1 - Python Input Reception	15
15.2 Test 2 - Wekinator Input Configuration	15
15.3 Test 3 - Wekinator Training	15
15.4 Test 4 - Processing Output Reception	15
16 Final Deliverables	15
17 Conclusion	17
References	17

1. Title

Hand Gesture Regression Using MediaPipe, OSC Communication, Wekinator, and Real-Time RGB Rectangle Visualization

2. Abstract

This report describes the implementation of **Lab 3 - Exercise 2: Gesture-based visual control**. The objective of the exercise was to build a real-time interactive machine learning system in which hand movement data controls continuous visual output. A Python input script uses MediaPipe to detect one hand from the webcam, extract hand landmark coordinates, and send those numerical features to Wekinator using OSC. Wekinator learns a supervised regression mapping from the hand landmark space to three continuous RGB output values. A Processing output sketch receives the predicted values from Wekinator and uses them to control the color of a rectangle in real time.

The final work contains three comparable experiments. The first experiment uses all **21 hand landmarks**, corresponding to **63 Wekinator inputs**. The second experiment uses **10 selected landmarks**, corresponding to **30 Wekinator inputs**. The third experiment uses **3 selected landmarks**, corresponding to **9 Wekinator inputs**. All experiments use the same output structure: **3 continuous outputs** for Red, Green, and Blue.

The complete system demonstrates regression because the model does not return a fixed class label. Instead, it continuously predicts numerical color values. This allows hand gestures to become expressive visual controls rather than discrete gesture categories.

3. Exercise Objective

The objective of Exercise 2 was to explore how body movement data can be used as input for a machine learning system and how regression can map physical hand gestures to continuous visual parameters.

The required task was:

Hand gesture landmarks -> Wekinator regression model -> RGB rectangle color

The exercise required a Python script to capture one hand using MediaPipe, extract landmark positions, and send those coordinates to Wekinator. It also required a Processing sketch to receive three output values from Wekinator and use them as RGB color channels for a rectangle.

The comparison required by the exercise was performed with three different input representations:

Experiment	Landmark positions	Coordinates per landmark	Wekinator inputs
Experiment 1	21	3	63
Experiment 2	10	3	30
Experiment 3	3	3	9

4. System Overview

The implemented system contains three main parts:

1. Python input script for webcam acquisition, MediaPipe hand detection, feature extraction, and OSC transmission.

2. Wekinator regression project for supervised learning and continuous output prediction.
3. Processing output sketch for receiving Wekinator RGB outputs and visualizing them as a colored rectangle.

The complete data flow is:

```
Webcam image
-> Python input script
-> MediaPipe hand detection
-> hand landmark coordinate extraction
-> OSC message /wek/inputs on port 6448
-> Wekinator regression model
-> 3 continuous RGB predictions
-> OSC message /wek/outputs on port 12000
-> Processing output sketch
-> rectangle color changes in real time
```

Component	Responsibility
Python input script	Captures webcam frames, detects one hand, extracts selected landmarks, and sends input features to Wekinator.
MediaPipe	Provides the hand skeleton landmarks used as numerical input data.
OSC	Connects Python to Wekinator and Wekinator to Processing using real-time messages.
Wekinator	Learns the regression mapping from hand coordinates to RGB values.
Processing output sketch	Receives RGB predictions and displays the resulting rectangle color.

5. Software and Libraries Used

The implementation used the following tools and libraries.

Tool / Library	Purpose
Python	Main language for webcam capture, MediaPipe processing, and OSC input transmission.
OpenCV / cv2	Reads webcam frames, flips the frame for natural interaction, and displays the camera window.
MediaPipe	Detects the hand and returns hand landmark coordinates.
python-osc	Sends the feature vector from Python to Wekinator.
Wekinator	Interactive machine learning software used for continuous regression.
Processing	Creative coding environment used for the output visualization sketch.
oscP5 / netP5	Processing libraries used to receive OSC messages from Wekinator.
OSC	Communication protocol used between the different programs.

6. Python Input Script

6.1 Purpose of the Python Script

The Python input script is the first script in the pipeline. Its purpose is to capture video from the webcam, detect one hand with MediaPipe, extract the selected landmark coordinates, and send those values to Wekinator through OSC.

The Python script does not perform the regression itself. Its role is data acquisition and feature extraction.

6.2 MediaPipe Hand Detection

The script uses MediaPipe's hand landmarker model:

```
MODEL_PATH = "hand_landmarker.task"
```

The MediaPipe options are configured for one hand:

```
options = vision.HandLandmarkerOptions(
    base_options=base_options,
    num_hands=1,
    min_hand_detection_confidence=0.7,
    min_hand_presence_confidence=0.7,
    min_tracking_confidence=0.7
)
```

The value `num_hands = 1` matches the exercise requirement because the system is based on the skeleton of a single hand.

6.3 Landmark Coordinate Extraction

Each MediaPipe hand landmark contains three coordinate values:

```
x coordinate
y coordinate
z coordinate
```

Therefore, the number of Wekinator inputs is calculated as:

```
number of selected landmarks x 3 coordinates
```

The project used the following three landmark modes:

```
LANDMARK_SETS = {
    21: list(range(21)),
    10: [0, 4, 5, 8, 9, 12, 13, 16, 17, 20],
    3: [0, 8, 20]
}
```

6.4 Landmark Modes

The three versions are summarized below.

Mode	Selected landmarks	Input count
21 landmarks	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20	63
10 landmarks	0, 4, 5, 8, 9, 12, 13, 16, 17, 20	30
3 landmarks	0, 8, 20	9

The selected landmarks are sent in order. For every selected landmark, the script appends the **x**, **y**, and **z** coordinates to the feature vector.

```
for index in selected_landmarks:
    lm = hand[index]
    features.append(float(lm.x))
    features.append(float(lm.y))
    features.append(float(lm.z))
```

6.5 OSC Output from Python to Wekinator

The Python script sends the extracted feature vector to Wekinator using the OSC address `/wek/inputs`.

Parameter	Value
Destination host	127.0.0.1 / localhost
Destination port	6448
OSC input message	/wek/inputs
Experiment 1 values per message	63
Experiment 2 values per message	30
Experiment 3 values per message	9

The OSC client is created as:

```
client = udp_client.SimpleUDPClient(WEKINATOR_HOST, WEKINATOR_PORT)
```

The features are sent using:

```
client.send_message(WEKINATOR_MESSAGE, features)
```

7. Wekinator Regression Project

7.1 Purpose of Wekinator

Wekinator is the learning component of the system. It receives landmark coordinates from the Python input script and learns a supervised regression mapping from hand shape to three output values.

The learned mapping is:

```
hand landmark coordinates -> Red + Green + Blue
```

The output values are continuous, not categorical. This means Wekinator can produce intermediate RGB values and the Processing rectangle can change color smoothly.

7.2 Wekinator Input Configuration

Each experiment used a separate Wekinator project because the number of inputs changes between the 21, 10, and 3 landmark versions.

Configuration	Experiment 1	Experiment 2	Experiment 3
Landmarks	21	10	3
Inputs	63	30	9
OSC input port	6448	6448	6448
OSC input message	/wek/inputs	/wek/inputs	/wek/inputs

The Wekinator project files confirm these input counts:

```
Experiment 1 inputConfig.xml -> <numInputs>63</numInputs>
Experiment 2 inputConfig.xml -> <numInputs>30</numInputs>
Experiment 3 inputConfig.xml -> <numInputs>9</numInputs>
```

7.3 Wekinator Output Configuration

All experiments used the same output configuration because the visual task was always RGB color control.

Parameter	Value
Number of outputs	3
Output 1	Red channel
Output 2	Green channel
Output 3	Blue channel
Output OSC message	/wek/outputs
Output host	localhost
Output port	12000
Output type	All continuous
Output range	0.0 to 1.0

The output type is continuous because RGB color channels are numerical values. A hand gesture is therefore mapped to a point in RGB color space rather than to a fixed label.

7.4 Training Target Colors

Five training gestures were recorded in each experiment. Each gesture was paired with an RGB target value in Wekinator.

Gesture	Output 1 / R	Output 2 / G	Output 3 / B	Color name	Used in
Gesture 1	1.0	0.0	0.0	Red	All experiments
Gesture 2	0.0	1.0	0.0	Green	All experiments
Gesture 3	0.0	0.0	1.0	Blue	All experiments
Gesture 4	1.0	1.0	0.0	Yellow	All experiments
Gesture 5	1.0	1.0	0.6	Warm light yellow	Experiment 1
Gesture 5	1.0	0.0	1.0	Purple / magenta	Experiments 2 and 3

The first four gestures provide a common reference across all experiments. The fifth gesture provides an additional mixed-color target in each set.

8. Processing Output Sketch

8.1 Purpose of the Output Sketch

The Processing output sketch is responsible for visualizing the Wekinator regression output. It does not train the model and does not capture hand input. It only receives three predicted values from Wekinator and maps them to the RGB channels of a rectangle.

8.2 OSC Input to Processing

The Processing sketch listens on port 12000 and receives the OSC message `/wek/outputs`.

```
oscP5 = new OscP5(this, 12000);
```

When an OSC message is received, the sketch reads the first three values:

```
redOutput = theOscMessage.get(0).floatValue();  
greenOutput = theOscMessage.get(1).floatValue();  
blueOutput = theOscMessage.get(2).floatValue();
```

8.3 Mapping Wekinator Outputs to RGB

The Wekinator values are expected to be normalized between 0.0 and 1.0. Processing maps them to the standard RGB range from 0 to 255.

```
redValue = map(redOutput, 0, 1, 0, 255);  
greenValue = map(greenOutput, 0, 1, 0, 255);  
blueValue = map(blueOutput, 0, 1, 0, 255);
```

The rectangle is drawn using:

```
fill(redValue, greenValue, blueValue);  
rect(width / 2, height / 2 + 45, 360, 220, 20);
```

This directly satisfies the required visualization part of the exercise: Wekinator's three continuous outputs control the Red, Green, and Blue channels of a rectangle.

9. Training Methodology

9.1 Training Concept

Training Wekinator means recording supervised examples. During recording, Python continuously sends the current landmark coordinates as inputs. At the same time, the desired RGB target values are manually selected in Wekinator using the output sliders.

Each recorded row has this structure:

```
ID, Time, Training round, inputs..., outputs-1, outputs-2, outputs-3
```

The input part changes with the selected landmark mode. The output part always contains the same three RGB values.

9.2 Training Workflow

The training workflow was the same for the three experiments:

1. Open the correct Wekinator project for the selected input count.
2. Start Wekinator listening on port 6448.
3. Run the Python script for the selected landmark mode.
4. Confirm that Wekinator is receiving the OSC input values.
5. Choose the RGB target value for the current gesture.
6. Perform the hand gesture and record examples.
7. Repeat the process for five gestures and five color targets.
8. Train the Wekinator regression model.
9. Click Run in Wekinator.
10. Run the Processing output sketch and observe the rectangle color.

9.3 Recorded Training Data Summary

The Wekinator ARFF files contain the recorded training examples. The following table reports the exact number of rows stored for each experiment.

Metric	Experiment 1	Experiment 2	Experiment 3
Landmarks	21	10	3
Inputs	63	30	9
Outputs	3	3	3
Training rounds	5	5	5
Recorded examples	1162	1289	1590
Average examples per round	232.4	257.8	318.0
Active recording duration	39.657 s	42.783 s	53.599 s
Average recording rate	29.30 rows/s	30.13 rows/s	29.66 rows/s

10. Experiment 1 - 21 Landmarks and 63 Inputs

10.1 Configuration

Experiment 1 used all 21 MediaPipe hand landmarks. Since each landmark contains **x**, **y**, and **z**, the resulting input vector contains 63 values.

```

LANDMARK_MODE = 21
selected_landmarks = [0, 1, 2, ..., 20]
input_count = 21 x 3 = 63

```

Property	Value
Landmark count	21
Wekinator inputs	63
Wekinator outputs	3
Input OSC message	/wek/inputs
Input port	6448
Output OSC message	/wek/outputs
Output port	12000
Training examples	1162
Training rounds	5

10.2 Training Data

Round	Color target	R	G	B	Examples
1	Red	1.0	0.0	0.0	220
2	Green	0.0	1.0	0.0	215
3	Blue	0.0	0.0	1.0	242
4	Yellow	1.0	1.0	0.0	249
5	Warm light yellow	1.0	1.0	0.6	236

10.3 Result

The 21-landmark version provided the most detailed representation of the hand because the full skeleton was used. Wekinator received information from the wrist, finger bases, intermediate finger joints, and fingertips. This produced a rich input space for learning the relationship between hand shape and RGB output.

The trained model used 1162 examples and produced real-time continuous output values for the Processing rectangle. The rectangle color updated according to the predicted Red, Green, and Blue values.

11. Experiment 2 - 10 Landmarks and 30 Inputs

11.1 Configuration

Experiment 2 used a reduced landmark set of 10 selected points. These landmarks were chosen to preserve important hand structure while reducing the number of inputs.

```

LANDMARK_MODE = 10
selected_landmarks = [0, 4, 5, 8, 9, 12, 13, 16, 17, 20]
input_count = 10 x 3 = 30

```

Property	Value
Landmark count	10
Wekinator inputs	30
Wekinator outputs	3
Input OSC message	/wek/inputs
Input port	6448
Output OSC message	/wek/outputs
Output port	12000
Training examples	1289
Training rounds	5

11.2 Training Data

Round	Color target	R	G	B	Examples
1	Red	1.0	0.0	0.0	231
2	Green	0.0	1.0	0.0	257
3	Blue	0.0	0.0	1.0	286
4	Yellow	1.0	1.0	0.0	247
5	Purple / magenta	1.0	0.0	1.0	268

11.3 Result

The 10-landmark version produced a compact but expressive hand representation. It kept selected landmarks from the wrist, thumb, index finger, middle finger, ring finger, and pinky. The input count was reduced from 63 to 30, while the output task remained unchanged with three continuous RGB values.

The trained model used 1289 examples. The real-time output continued to update the Processing rectangle color through the same `/wek/outputs` message on port 12000.

12. Experiment 3 - 3 Landmarks and 9 Inputs

12.1 Configuration

Experiment 3 used a minimal landmark set containing the wrist, index fingertip, and pinky fingertip.

```

LANDMARK_MODE = 3
selected_landmarks = [0, 8, 20]
input_count = 3 x 3 = 9

```

Property	Value
Landmark count	3
Wekinator inputs	9
Wekinator outputs	3
Input OSC message	/wek/inputs
Input port	6448
Output OSC message	/wek/outputs
Output port	12000
Training examples	1590
Training rounds	5

12.2 Training Data

Round	Color target	R	G	B	Examples
1	Red	1.0	0.0	0.0	277
2	Green	0.0	1.0	0.0	270
3	Blue	0.0	0.0	1.0	333
4	Yellow	1.0	1.0	0.0	382
5	Purple / magenta	1.0	0.0	1.0	328

12.3 Result

The 3-landmark version provided the smallest input representation. It used only 9 Wekinator inputs while keeping exactly the same RGB output structure. This made the experiment useful for observing how a very compact hand skeleton can still drive a continuous visual output.

The trained model used 1590 examples, the largest training set among the three experiments. After training, the Processing rectangle continued to respond to Wekinator's continuous RGB predictions.

13. Detailed Recorded Data Comparison

13.1 Input and Output Comparison

Experiment	Landmarks	Inputs	Outputs	Input reduction vs. 21	Examples	Rounds
1	21	63	3	0.0%	1162	5
2	10	30	3	52.38%	1289	5
3	3	9	3	85.71%	1590	5

The input reduction is calculated from the original 63-input version:

```
Experiment 2 reduction = (63 - 30) / 63 = 52.38%
Experiment 3 reduction = (63 - 9) / 63 = 85.71%
```

13.2 Training Example Distribution

Experiment	Round 1	Round 2	Round 3	Round 4	Round 5
21 landmarks / 63 inputs	220	215	242	249	236
10 landmarks / 30 inputs	231	257	286	247	268
3 landmarks / 9 inputs	277	270	333	382	328

13.3 Training Duration and Recording Rate

Experiment	Active recording duration	Recorded examples	Average recording rate
21 landmarks / 63 inputs	39.657 s	1162	29.30 rows/s
10 landmarks / 30 inputs	42.783 s	1289	30.13 rows/s
3 landmarks / 9 inputs	53.599 s	1590	29.66 rows/s

All three experiments recorded data at approximately 30 rows per second. This confirms that the three Python input scripts sent continuous real-time feature streams to Wekinator.

13.4 Wekinator Data and Model File Sizes

The stored project files also show how the input representation affects the amount of stored data.

Experiment	ARFF training file	Sum of 3 model files	Interpretation
21 landmarks / 63 inputs	713,199 bytes	2,839,091 bytes	Full-size input representation
10 landmarks / 30 inputs	402,874 bytes	1,384,105 bytes	Medium-size input representation
3 landmarks / 9 inputs	189,884 bytes	606,787 bytes	Compact input representation

The three experiments keep the same number of outputs, but the number of stored input attributes changes from 63 to 30 to 9. This explains the decreasing size of the Wekinator training and model files.

14. Performance Analysis and Discussion

14.1 Regression Behavior

The system demonstrates regression because Wekinator predicts continuous numerical values. The output is not a discrete class such as “red gesture” or “blue gesture”. Instead, Wekinator outputs three real numbers:

```
outputs-1 -> Red
outputs-2 -> Green
outputs-3 -> Blue
```

These values can take intermediate values between 0.0 and 1.0. Processing maps them to the RGB range between 0 and 255. This creates smooth visual control of the rectangle color.

14.2 Experiment 1: 21 Landmarks

The 21-landmark experiment gave Wekinator the most complete hand skeleton. It used all landmark positions returned by MediaPipe. This version provided the highest amount of hand-shape information: 63 continuous inputs per frame.

Because the full hand structure was available, this experiment represented the most detailed version of the system. It is the best version when the goal is to preserve maximum information about finger shape, palm structure, and hand configuration.

14.3 Experiment 2: 10 Landmarks

The 10-landmark experiment reduced the input size from 63 to 30 values. This is a reduction of 52.38% while still keeping important points from different parts of the hand.

This version provided a balanced representation. It kept the wrist, fingertips, and selected finger joints, so the model still received meaningful shape information while using fewer inputs than the full 21-landmark version.

14.4 Experiment 3: 3 Landmarks

The 3-landmark experiment reduced the input size from 63 to 9 values. This is a reduction of 85.71%. It used the wrist, index fingertip, and pinky fingertip as a compact geometric representation of the hand.

This version was the lightest experiment in terms of input size and stored model size. It is useful as a compact baseline for comparing how much visual control can be produced from a very small number of hand points.

14.5 Overall Comparison

Aspect	21 landmarks		10 landmarks		3 landmarks	
Input detail	Highest	detail:	Balanced	detail:	Compact	detail:
	complete	hand	selected	structural	wrist and two fin-	
	skeleton		points		gertips	
Input count	63		30		9	
Training examples	1162		1289		1590	
Stored ARFF size	713,199 bytes		402,874 bytes		189,884 bytes	
Model storage	2,839,091 bytes		1,384,105 bytes		606,787 bytes	
Best use in this comparison	Maximum	hand-	Balanced	represen-	Minimal	represen-
	shape	representa-	tation		tation	
	tion					

The comparison shows that reducing the number of landmarks directly reduces the number of Wekinator inputs and the amount of stored model data. The visual output task remains the same for all three experiments because every model still predicts the same three continuous RGB channels.

15. Testing and Verification

The final system was verified through the following checks.

15.1 Test 1 - Python Input Reception

The Python script was run for each landmark mode. The webcam window displayed the detected hand and highlighted the selected landmarks.

Expected behavior:

```
The hand is detected.
The selected landmarks are drawn in green.
The status text reports the correct input count.
```

This confirmed that MediaPipe and the landmark selection logic were working.

15.2 Test 2 - Wekinator Input Configuration

Each Wekinator project was checked with the matching input count.

Experiment	Python sends	Wekinator expects
21 landmarks	63 values	63 inputs
10 landmarks	30 values	30 inputs
3 landmarks	9 values	9 inputs

This confirmed that the Python feature vector and Wekinator configuration matched for each experiment.

15.3 Test 3 - Wekinator Training

Each Wekinator project stored five training rounds. The ARFF files confirm that the training data was recorded successfully:

```
Experiment 1 -> 1162 examples
Experiment 2 -> 1289 examples
Experiment 3 -> 1590 examples
```

Each row contains an input feature vector and the corresponding three RGB target outputs.

15.4 Test 4 - Processing Output Reception

The Processing sketch listened on port 12000 and received the `/wek/outputs` message from Wekinator.

Expected behavior:

```
Output 1 - Red raw value changes.
Output 2 - Green raw value changes.
Output 3 - Blue raw value changes.
Mapped R, G, and B values update between 0 and 255.
The rectangle color changes in real time.
```

This confirmed that the output sketch was receiving Wekinator predictions correctly.

16. Final Deliverables

The completed Exercise 2 submission contains the following materials.

Deliverable	Included material
Experiment 1	Python input script, Processing output sketch, Wekinator project for 21 landmarks / 63 inputs.
Experiment 2	Python input script, Processing output sketch, Wekinator project for 10 landmarks / 30 inputs.
Experiment 3	Python input script, Processing output sketch, Wekinator project for 3 landmarks / 9 inputs.
Final report	Explanation of the system, training method, exact numbers, results, and comparison.

The recommended folder structure is:

```

Lab3_Exercise2/

Experiment_1_21_Landmarks_63_Inputs/
  Python_Input/
    script.py
    hand_landmarker.task
  Processing_Output/
    ColorOutput.pde
  Wekinator_Project/
    WekinatorProject.wekproj
    inputConfig.xml
    outputConfig.xml
    current/currentData.arff
    current/models/

Experiment_2_10_Landmarks_30_Inputs/
  Python_Input/
    script.py
    hand_landmarker.task
  Processing_Output/
    ColorOutput.pde
  Wekinator_Project/
    WekinatorProject.wekproj
    inputConfig.xml
    outputConfig.xml
    current/currentData.arff
    current/models/

Experiment_3_3_Landmarks_9_Inputs/
  Python_Input/
    script.py
    hand_landmarker.task
  Processing_Output/
    ColorOutput.pde
  Wekinator_Project/
    WekinatorProject.wekproj
    inputConfig.xml
    outputConfig.xml
    current/currentData.arff
    current/models/

Final_Report/
  Final_Report.pdf
  Final_Report.tex

```

17. Conclusion

This exercise implemented a complete real-time gesture-based visual control system using Python, MediaPipe, OSC, Wekinator, and Processing. The Python script captured the webcam stream, detected one hand, extracted selected landmark coordinates, and sent them to Wekinator as numerical input features. Wekinator learned a supervised regression mapping from hand landmark positions to three continuous RGB output values. The Processing output sketch received these predictions and displayed the result as a colored rectangle that changed in real time.

The work was completed through three experiments. Experiment 1 used 21 landmarks and 63 inputs, Experiment 2 used 10 landmarks and 30 inputs, and Experiment 3 used 3 landmarks and 9 inputs. All experiments used 3 continuous outputs for RGB control and 5 training rounds. The recorded training sets contained 1162 examples for the 21-landmark version, 1289 examples for the 10-landmark version, and 1590 examples for the 3-landmark version.

The comparison shows that the full 21-landmark setup provides the richest hand representation, the 10-landmark setup provides a balanced reduced representation, and the 3-landmark setup provides the most compact representation. Across all three experiments, the same real-time regression pipeline remained valid:

`Hand movement -> landmark coordinates -> Wekinator regression -> RGB rectangle color`

The final result satisfies the exercise goal by demonstrating how physical hand gestures can be transformed into continuous visual parameters through interactive machine learning.

References

1. Lab 3 assignment PDF, *Music and multimedia streaming over the Internet*, Politecnico di Torino, Academic year 2025/2026.
2. MediaPipe Python library.
3. Processing language reference.
4. oscP5 Processing library.
5. Python language documentation.